Лабораторная работа 2
«Предобработка датасета»

Гузовская Александра Чеславовна
Б9123-01.03.02-сп

17 октября 2026 г.

Описание выбранного датасета

Датасет представляет собой 33 колонки: время заполнения анкеты, возраст, любимый жанр, частоты прослушивания жанров, ментальные расстройства от 0 до 10, скорость треков и согласие на участие

Проблемы для предобработки

Есть пропущенные значения, чаще всего до 10, но в столбце BPM 107 пропусков (около 1/6 датасета)
Есть выбросы, особенно в BPM, значения 999999999 и 624 значительно влияют на среднее значение

Этапы предобработки с описанием

1) Обработка пропусков
(выводим колонки со всеми их возможными уникальными значениями и встречаемостью значений, смотрим на количество пропусков) - малые пропуски заполняем либо рандомно (для да/нет),

Добавление и удаление столбцов с обоснованием

Первым делом посмотрели, что в согласии на участие нет пропусков и доступно только одно значение, значит это кандидат на удаление
Время заполнения анкеты можно было бы не удалять, если бы от времени заполнения анкеты что-то бы зависело. Но не указан ни город, ни часовой пояс респондентов, а время (предположительно) берётся у сервера. От сезона заполнения анкеты особо тоже ничего не зависит

Код программы

```
import pandas as pd
import numpy as np

df = pd.read_csv('music_mental.csv')

print("\nПРОВЕРКА ПРОПУСКОВ И ДУБЛИКАТОВ")
missing = df.isnull().sum()
missing = missing[missing > 0]
if len(missing) > 0:
    print("Найдены пропуски:")
    print(missing)
else:
    print("Пропусков нет")

df_check = df.drop(columns=['Timestamp'])

full_duplicates = df_check.duplicated().sum()
print(f"Полных дубликатов строк (без учета Timestamp): {full_duplicates}")

with open('analysis_output.txt', 'w', encoding='utf-8') as f:
    f.write("Колонки и их значения\n")
    for col in df.columns:
        f.write(f"\n {col}\n")

        values = df[col].value_counts(dropna=False)
        for val, count in values.items():
            if pd.isna(val):
                f.write(f"  NaN (пропуск): {count} раз\n")
            else:
                f.write(f"  {val}: {count} раз\n")

print("\nПРОВЕРКА ВЫБРОСОВ (ЧИСЛОВЫЕ КОЛОНКИ)")
numeric_cols = ['Age', 'Hours per day', 'BPM', 'Anxiety', 'Depression', 'Insomnia', 'OCD

for col in numeric_cols:
    if col in df.columns:
        print(f"\n{col}: Min: {df[col].min()}, Max: {df[col].max()}, Mean: {df[col].mean
        q25 = df[col].quantile(0.25)
        q50 = df[col].quantile(0.5)
        q75 = df[col].quantile(0.75)
        print(f"  Квантили: 25%={q25}, 50%={q50}, 75%={q75}")

        if col == 'Age':
            problematic = df[(df[col] < 10) | (df[col] > 80)]
            if len(problematic) > 0:
                print(f"Подозрительный возраст (<10 или >80): {problematic[col].tolist()

        if col == 'Hours per day':
```

```
            problematic = df[df[col] > 16]
            if len(problematic) > 0:
                print(f"Часов в день больше 16: {problematic[col].tolist()}")

        if col == 'BPM':
            problematic = df[(df[col] < 60) | (df[col] > 200)][col]
            problematic_0 = df[(df[col] < 10)][col]
            if len(problematic) > 0:
                print(f"Подозрительный BPM (<60 или >200): {problematic.tolist()}")
            if len(problematic_0) > 0:
                print(f"Почти нулевой BPM (<10): {problematic_0.tolist()}")

        if col in ['Anxiety', 'Depression', 'Insomnia', 'OCD']:
            problematic = df[(df[col] < 0) | (df[col] > 10)]
            if len(problematic) > 0:
                print(f"Значения вне диапазона 0-10: {problematic[col].tolist()}")

print("\nОБРАБОТКА ПРОПУСКОВ")
age_median = round(df['Age'].median())
df['Age'] = df['Age'].fillna(age_median)

df['Primary streaming service'] = df['Primary streaming service'].fillna('Other')

if df['While working'].isnull().sum() > 0:
    random_values = np.random.choice(['Yes', 'No'], size=df['While working'].isnull().su
    df.loc[df['While working'].isnull(), 'While working'] = random_values

df['Instrumentalist'] = df['Instrumentalist'].fillna('No')

df['Composer'] = df['Composer'].fillna('No')

if df['Foreign languages'].isnull().sum() > 0:
    random_values = np.random.choice(['Yes', 'No'], size=df['Foreign languages'].isnull(
    df.loc[df['Foreign languages'].isnull(), 'Foreign languages'] = random_values

df['Music effects'] = df['Music effects'].fillna('No effect')

bpm_05 = df['BPM'].quantile(0.05)
bpm_95 = df['BPM'].quantile(0.95)
print(f"5-й процентиль BPM: {bpm_05:.1f}")
print(f"95-й процентиль BPM: {bpm_95:.1f}")

problematic_bpm = df[(df['BPM'].isnull()) | (df['BPM'] < bpm_05) | (df['BPM'] > bpm_95)]
print(f"Найдено проблемных BPM: {len(problematic_bpm)}")

for idx in problematic_bpm:
    genre = df.loc[idx, 'Fav genre']
    genre_median = df[df['Fav genre'] == genre]['BPM'].median()
    df.loc[idx, 'BPM'] = round(genre_median) if not pd.isna(genre_median) else round(df[
```

```
print("\nОстались ли пропуски после обработки:")
print(df.isnull().sum()[df.isnull().sum() > 0])

freq_map = {
    'Never': 0,
    'Rarely': 1,
    'Sometimes': 2,
    'Very frequently': 3
}
freq_cols = [col for col in df.columns if 'Frequency' in col]
for col in freq_cols:
    df[col] = df[col].map(freq_map)

print("\nДОБАВЛЕНИЕ СТОЛБЦОВ (ПРИЗНАКОВ)")

def age_category(age):
    if age < 13:
        return 'children'
    elif age < 20:
        return 'teens'
    elif age < 30:
        return 'twenties'
    elif age < 40:
        return 'thirties'
    elif age < 50:
        return 'forties'
    elif age < 65:
        return 'pre_retirement'
    else:
        return 'retired'
df['age_category'] = df['Age'].apply(age_category)
print("1. Добавлен age_category - возрастная категория")
# print("Распределение по возрастным категориям:")
# print(df['age_category'].value_counts().sort_index())

df['mental_sum'] = df[['Anxiety', 'Depression', 'Insomnia', 'OCD']].sum(axis=1)
print("2. Добавлен mental_sum - сумма всех расстройств")

calm_genres = ['Frequency [Lofi]', 'Frequency [Classical]', 'Frequency [Jazz]']
df['prefers_calm'] = df[calm_genres].apply(lambda x: sum([1 for v in x if v >= 2]), axis
print("3. Добавлен prefers_calm - счетчик спокойных жанров")

energetic_genres = ['Frequency [Metal]', 'Frequency [EDM]', 'Frequency [Hip hop]']
df['prefers_energetic'] = df[energetic_genres].apply(lambda x: sum([1 for v in x if v >=
)
print("4. Добавлен prefers_energetic - счетчик энергичных жанров")

freq_cols = [col for col in df.columns if 'Frequency' in col]
genre_names = [col.replace('Frequency [', '').replace(']', '') for col in freq_cols]
```

```
def get_avg_bpm_favorite(row):
    bpms = []

    fav_genre = row['Fav genre']
    fav_genre_bpm = df[df['Fav genre'] == fav_genre]['BPM'].mean()
    bpms.append(fav_genre_bpm)

    for genre_col, genre_name in zip(freq_cols, genre_names):
        if row[genre_col] == 3:
            genre_bpm = df[df['Fav genre'] == genre_name]['BPM'].mean()
            bpms.append(genre_bpm)

    return round(np.mean(bpms), 1)

df['avg_bpm_favorite'] = df.apply(get_avg_bpm_favorite, axis=1)
print("5. Добавлен avg_bpm_favorite - средний BPM любимого жанра + Very frequently жанро

def bpm_category(bpm):
    if pd.isna(bpm):
        return 'unknown'
    elif bpm < 60:
        return 'very_slow'
    elif bpm < 85:
        return 'slow'
    elif bpm < 120:
        return 'moderate'
    elif bpm < 150:
        return 'fast'
    else:
        return 'extremely_fast'
df['bpm_category'] = df['BPM'].apply(bpm_category)
print("6. Добавлен bpm_category - категория BPM")

df = df.drop(columns=['Timestamp', 'Permissions'])
print("\nУдалены колонки Timestamp и Permissions")

print("\nДатасет обработан.")
print(f"Размер датасета: {df.shape}")

df.to_csv('music_mental_health_clean.csv', index=False)
```